RAG Flow on Azure Prompt Flow

---

1. Create an Azure OpenAI Resource:

1. In the Azure Portal, click **"+ Create a resource"**.
2. Search for **"Azure OpenAI"** and click on it → click **"Create"**.
3. Fill in:
   - Subscription: Your subscription.
   - Resource group: rag-promptflow (the one you just created).
   - Region: Same as resource group (e.g., East US).
   - Name: e.g., openai-rag-instance (must be globally unique).
   - Pricing tier: Standard
4. Click **"Next"** through the networking and tags tabs (defaults are fine).
5. Click **"Review + Create"** → **"Create"**.
6. Deployment takes about 2–3 minutes. Click **"Go to resource"** when done.

---

2. Deploy models inside Azure OpenAI Studio:

1. On the resource page, click **"Go to Azure OpenAI Studio"** (button at the top).
2. In OpenAI Studio left menu, click **"Deployments"** → **"+ Create new deployment"**.
3. Deploy the **Chat model**:
   - Model: gpt-4.1-mini (or gpt-35-turbo if gpt-4o is unavailable)
   - Deployment name: gpt-4.1-mini ← note this name
   - Click **"Create"**.
4. Click **"+ Create new deployment"** again for the **Embedding model**:
   - Model: text-embedding-ada-002
   - Deployment name: embedding-ada-002 ← note this name
   - Click **"Create"**.
5. Go back to the Azure Portal resource page → click **"Keys and Endpoint"** in the left menu.
6. Copy and save:
   - **Key 1** (your API key)
   - **Endpoint** (e.g., https://openai-rag-instance.openai.azure.com/)

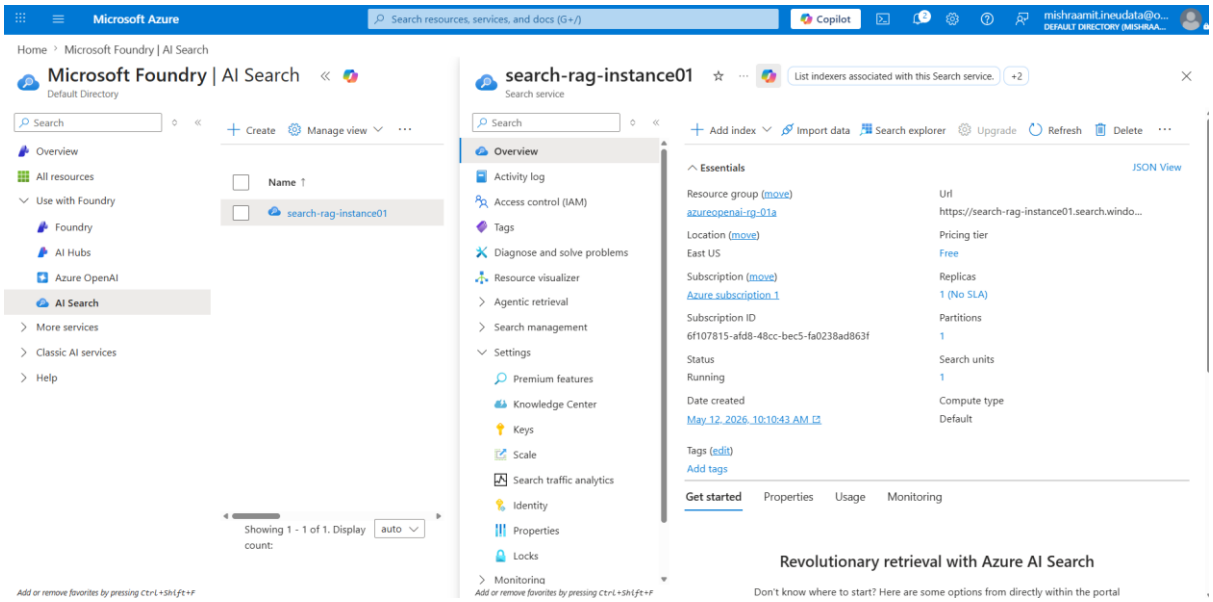
---

3. Create an Azure AI Search Resource:

Azure AI Search will act as your vector store.

1. In the Azure Portal, click **"+ Create a resource"** → search **"Azure AI Search"** → click **"Create"**.
2. Fill in:
   - **Subscription:** Your subscription.
   - **Resource group:** rag-prompt-flow.
   - **Service name:** e.g., search-rag-instance (must be globally unique, lowercase).
   - **Location:** Same region as your OpenAI resource.
   - **Pricing tier:** Basic (sufficient for development) or Standard for production.
3. Click **"Review + Create"** → **"Create"**. Deployment takes 2–5 minutes.
4. Once deployed, click **"Go to resource"**.

5. On the Overview page, copy and save:
    - **URL** (e.g., https://search-rag-instance.search.windows.net)
6. In the left menu, click **"Keys"** → copy and save **Primary admin key**.



---

5. Create an Azure Machine Learning Workspace:

Prompt Flow lives inside Azure Machine Learning (AML) Studio.

1. In the Azure Portal, click "+ **Create a resource**" → search "**Azure Machine Learning**" → click "**Create**".
2. Fill in:
    - **Subscription:** Your subscription.
    - **Resource group:** rag-promptflow.
    - **Workspace name:** e.g., aml-rag-workspace.
    - **Region:** East US.
    - Storage account, Key vault, Application insights: Leave as auto-create defaults.
    - **Container registry:** Leave as None (auto-created when needed).
3. Click "**Review + Create**" → "**Create**". This takes 3–5 minutes.
4. Once deployed, click "**Launch studio**" — this opens https://ml.azure.com.
5. Select your workspace from the dropdown if prompted.

Set Up Prompt Flow Environment in AML Studio

6. Create Connections (Securely Store API Keys):

Connections let Prompt Flow securely reference your API keys without hardcoding them.

Create Azure OpenAI Connection:

1. Click **"Connections"** tab → **"+ Create"** → choose **"Azure OpenAI"**.
2. Fill in:
    - **Name**: openai-connection
    - **API Key**: Paste the Key 1 (from azure openai model deployment)

- **API Base / Endpoint**: Paste the endpoint URL
- **API Type**: azure
- **API Version**: 2024-02-01 (use the latest stable version).

3. Click **"Save"**.

---

7. Create Azure AI Search Connection:

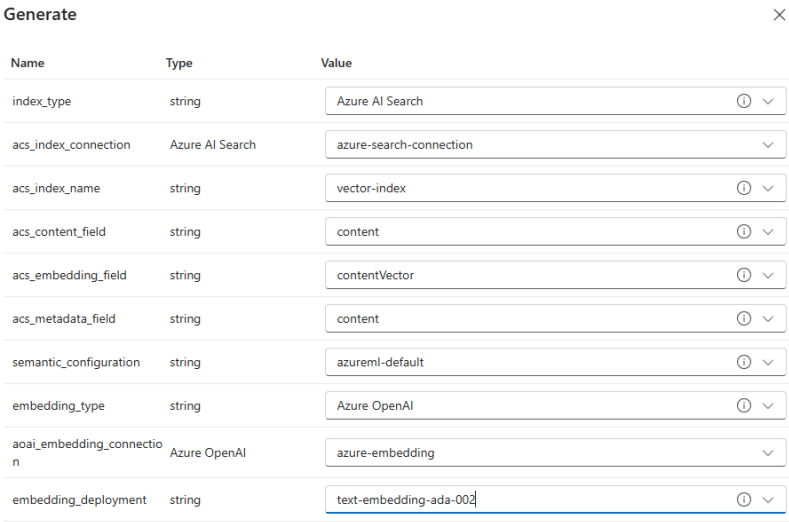
4. Click **"+ Create"** again → choose **"Cognitive Search"** (this is Azure AI Search).

5. Fill in:

   - **Name**: search-connection
   - **API Key**: Paste the Primary admin key from Step 4.
   - **API Base**: Paste the Search URL from Step 4 (e.g., https://search-rag-instance.search.windows.net).

6. Click **"Save"**.

Both connections now appear in your connections list and are available to all flows in this workspace.



Generate

Name	Type	Value
index_type	string	Azure AI Search
acs_index_connection	Azure AI Search	azure-search-connection
acs_index_name	string	vector-index
acs_content_field	string	content
acs_embedding_field	string	contentVector
acs_metadata_field	string	content
semantic_configuration	string	azureml-default
embedding_type	string	Azure OpenAI
aoai_embedding_connection	Azure OpenAI	azure-embedding
embedding_deployment	string	text-embedding-ada-002

---

8. Start a Compute Session (Runtime)

- In the Prompt Flow section, click **"Start Compute Session"**
- Choose **"Compute instance runtime"** for a dedicated VM, or **"Serverless (preview)"** for auto-managed compute.
- For Compute instance: choose an existing compute instance (or create one — Standard_DS3_v2 is fine for dev).

Prepare and Create Vector Index of Your Document:

9. Create a Vector Index Using the Built-in Wizard (Local File)

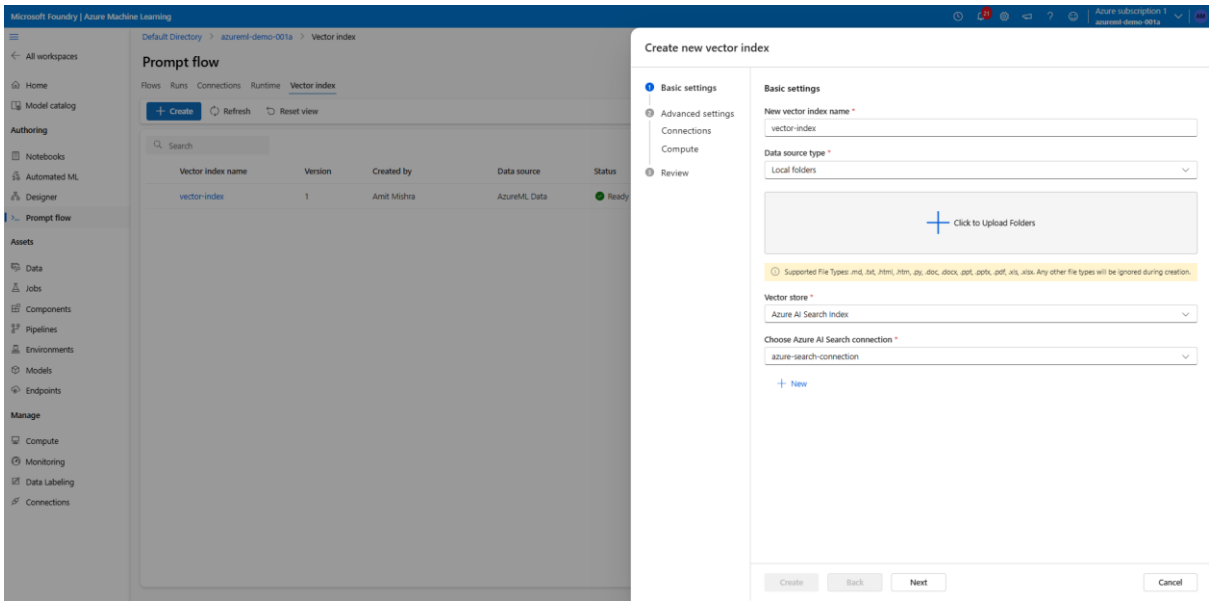
1. In **ml.azure.com** → left sidebar under **Authoring** → click **"Prompt Flow"**

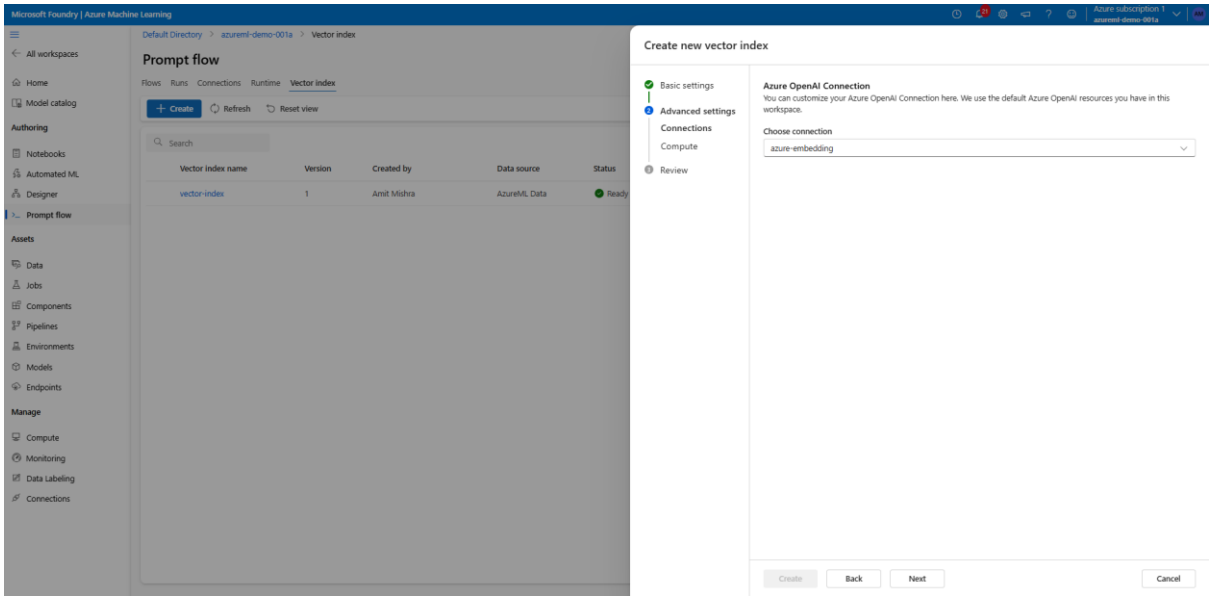
2. Click the **"Vector index"** tab at the top (you can see it in your screenshot next to Flows, Runs, Connections, Runtime)

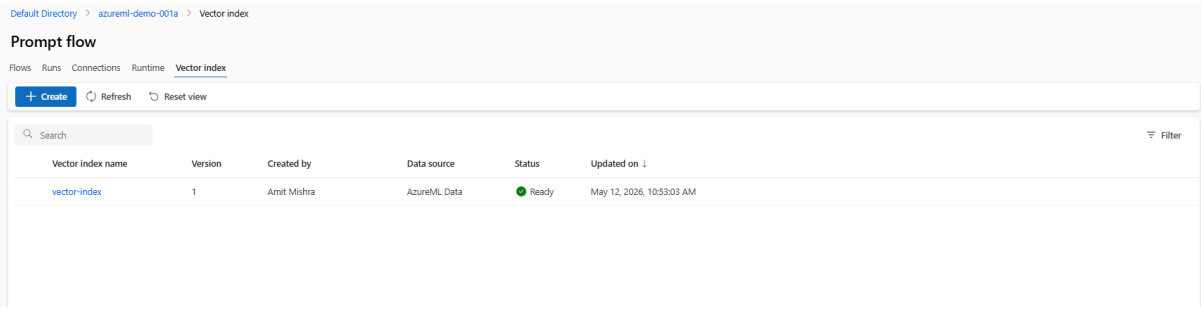
3. Click the **"+ Create"** button at the top left

4. **New vector index name**: Type your index name, e.g. rag-vector-index (replaces the default vector-index)
5. **Data source type**: Select **"Local folders"** from the dropdown
6. Click the **large upload box** that says **"Click to Upload Folders"**. Browse to your local folder containing your Tesla PDF/DOCX files → select the folder → click **Open**
7. Wait for the files to appear listed under the upload box.
8. **Vector store**: Confirm **"Azure AI Search Index"** is selected in the dropdown

9. **Choose Azure AI Search connection**: Select **azure-search-connection** from the dropdown
10. If you do not see a connection listed, click **"+ New"** (the button just below the dropdown) → fill in your Search endpoint URL and API key → Save. Click **"Next"** to proceed.







Build the RAG Flow Step by Step:

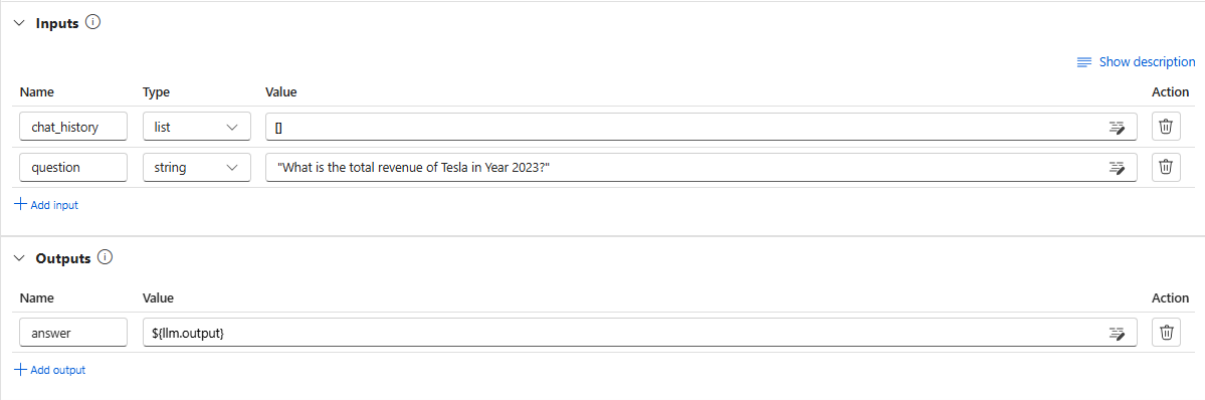
10. Create a New Standard Flow

1. In the Prompt Flow left sidebar, click **"Flows"** → **"+ Create"**.
2. Select **"Standard flow"** (not Chat flow).
3. **Name**: rag-flow → click **"Create"**.
4. The flow canvas opens. You'll see a default flow with: inputs → LLM → outputs. You will replace all of this.
5. At the top right of the canvas, click **"Start compute session"** → select your runtime → wait for it to show **Running**.

---

11. Configure Flow Inputs

1. Click the **"Inputs"** section at the top of the flow canvas.
2. Delete any existing default inputs.
3. Click **"+ Add input"**:
   - **Name**: question
   - **Type**: string
   - **Default value**: What is the refund policy? (or any test question relevant to your docs)
4. Click **"+ Add input"** again (optional, for multi-turn chat):
   - **Name**: chat_history
   - **Type**: list
   - **Default value**: []
5. Save the inputs.



---

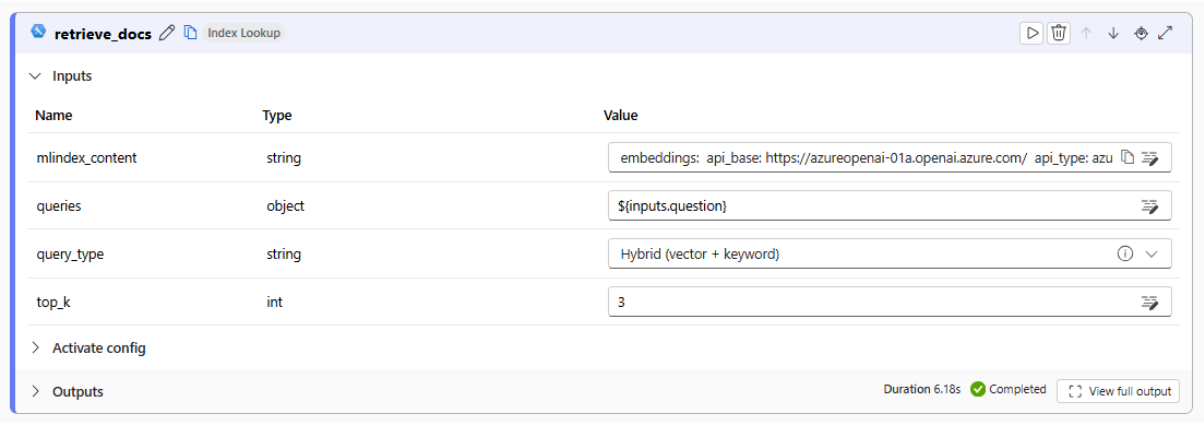
12. Retrieve Relevant Document Chunks

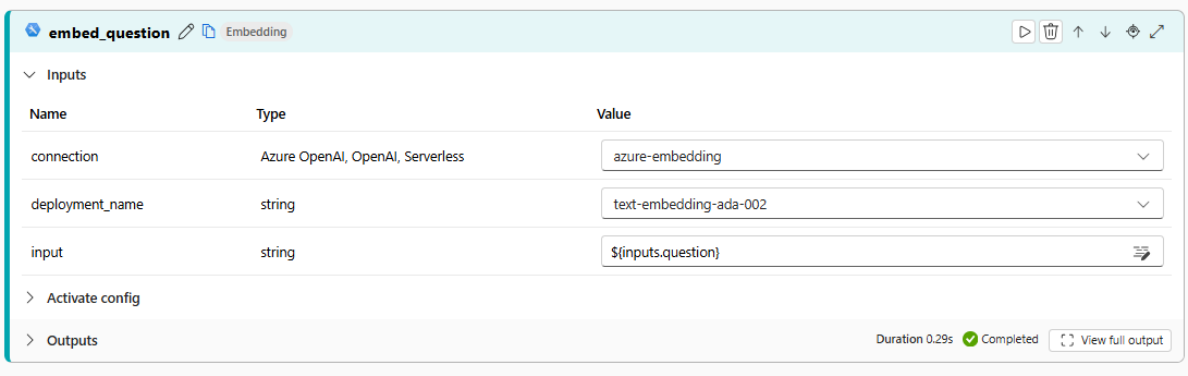
1. Click **"+ Add node"** → choose **"Vector DB Lookup"** (or search for **"Azure AI Search"** in the tool list).
2. **Name the node**: retrieve_docs.
3. In node settings:
   - **Connection**: search-connection
   - **Index name**: rag-vector-index
   - **Query**: ${inputs.question} (the raw text query for hybrid search)
   - **Query type**: Hybrid (uses both keyword + vector)
   - **Vector query**: ${embed_question.output} (the embedding from Node 1)

- **Top K**: 5 (retrieve the 5 most relevant chunks)
- **Select fields to return**: check text, sourcefile, id (or whatever fields your index has)

4. The output of this node is a list of document chunks (each with text + metadata).

5. Save the node.





---

13. Format the Retrieved Context

This step formats the retrieved chunks into clean readable text for the prompt.

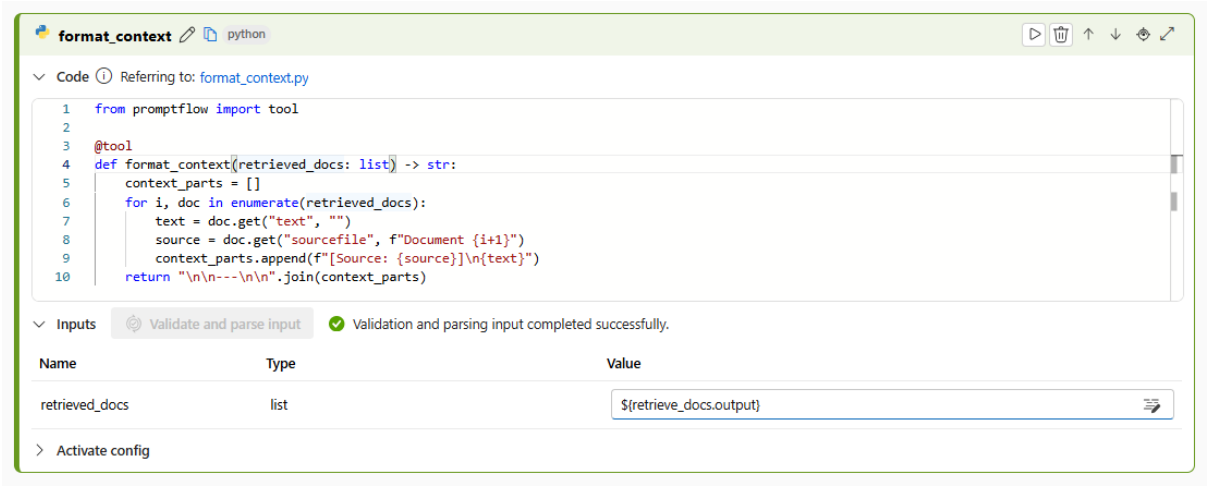
1. Click **"+ Add node"** → choose **"Python"**.

2. **Name the node**: format_context.

3. In the Python editor, replace the default code with:

```
from promptflow import tool
@tool
def format_context(retrieved_docs: list) -> str:
    context_parts = []
    for i, doc in enumerate(retrieved_docs):
        text = doc.get("text", "")
        source = doc.get("sourcefile", f"Document {i+1}")
        context_parts.append(f"[Source: {source}]\n{text}")
    return "\n\n---\n\n".join(context_parts)
```

4. Set inputs:
   - retrieved_docs: ${retrieve_docs.output}
5. Save the node. Click "Validate" to check for syntax errors.



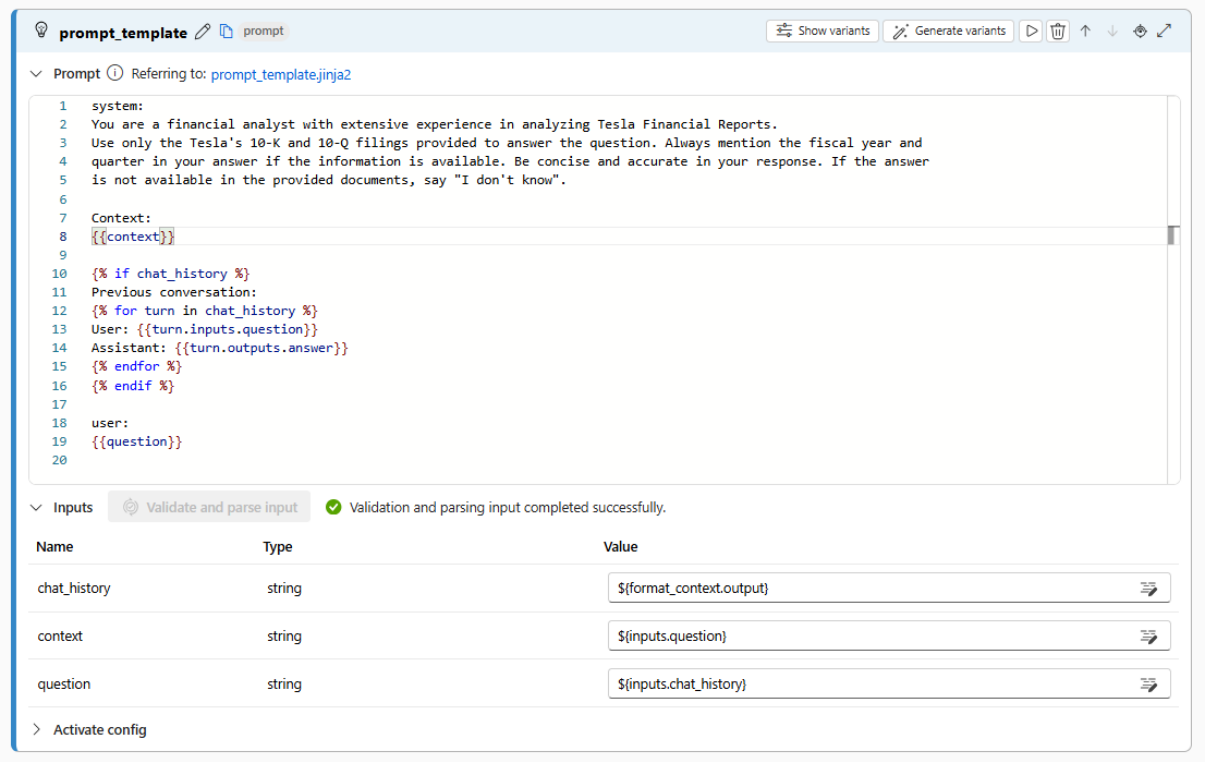
---

14. Build the RAG Prompt

1. Click **"+ Add node"** → choose **"Prompt"**.
2. **Name the node**: build_prompt.
3. In the prompt template editor, write the following template:

```
system:

You are a financial analyst with extensive experience in analyzing Tesla Financial
Reports. Use only the Tesla's 10-K and 10-
Q filings provided to answer the question. Always mention the fiscal year and
quarter in your answer if the information is available. Be concise and accurate in
your response. If the answer
is not available in the provided documents, say "I don't know".

Context:

{{context}}


{% if chat_history %}

Previous conversation:

{% for turn in chat_history %}

User: {{turn.inputs.question}}

Assistant: {{turn.outputs.answer}}

{% endfor %}

{% endif %}


user:

{{question}}
```

4. Set inputs:
   - **context**: ${format_context.output}
   - **question**: ${inputs.question}
   - **chat_history**: ${inputs.chat_history}
5. Save the node.



---

15. Generate the Answer with Azure OpenAI

1. Click **"+ Add node"** → choose **"LLM"** → choose **"AzureOpenAI"**.
2. **Name the node**: generate_answer.
3. In node settings:
   - **Connection**: openai-connection
   - **Deployment**: gpt-4o-deployment
   - **API**: Chat
   - **Prompt**: Click the link icon → select ${build_prompt.output}
4. Set model parameters:
   - **max_tokens**: 1000
   - **temperature**: 0.0 (keep it deterministic for factual RAG)
   - **top_p**: 1.0
   - **presence_penalty**: 0
   - **frequency_penalty**: 0
5. Save the node.

---

16. Configure Flow Outputs

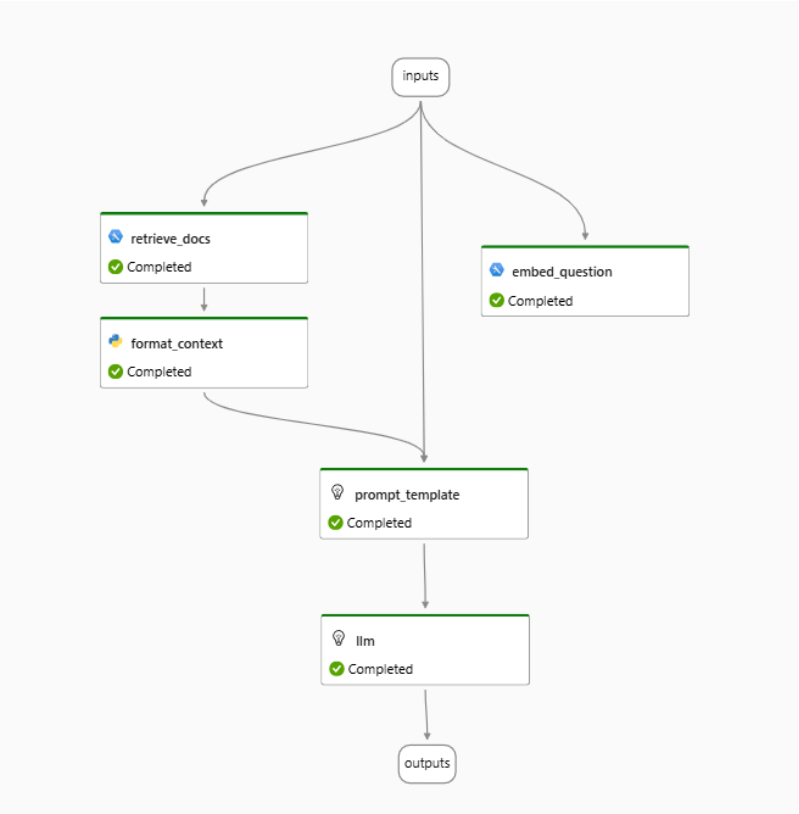
1. Click the **"Outputs"** section at the bottom of the canvas.
2. Delete any default outputs.
3. Click **"+ Add output"**:
   - **Name**: answer
   - **Type**: string
   - **Value**: ${generate_answer.output}

4. Optionally add a second output for traceability:

   - **Name**: retrieved_sources
   - **Type**: list
   - **Value**: ${retrieve_docs.output}

5. Save the outputs.

---

17. Verify the Flow Graph

- In the canvas, check that the node connections look like this, in order